



Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir
M.V. College
Of Science & Commerce

Practical 9: Linear Probing (Hash), Breadth-First
Tree, and Depth-First Tree using Graph

AIM:

9. A. linear probing (hash)

CODE:

```
def linear_probing_insert(hash_size, data):  
    hash_table = [None] * hash_size  
  
    for value in data:  
        hash_index = value % hash_size  
        original_index = hash_index  
  
        while hash_table[hash_index] is not None:  
            hash_index = (hash_index + 1) % hash_size  
            if hash_index == original_index:  
                print("Hash table is full!")  
                return hash_table  
  
        hash_table[hash_index] = value  
  
    return hash_table  
  
hash_size = int(input("Enter size of hash table: "))  
data_input = input("Enter data (space-separated numbers): ")  
data = list(map(int, data_input.split()))  
  
final_table = linear_probing_insert(hash_size, data)  
  
print("\nFinal Hash Table:")  
for i, v in enumerate(final_table):  
    print(f"Index {i}: {v}")
```

GUI CODE:

```
import tkinter as tk  
from tkinter import messagebox
```

```
def insert_all():
```

NAME: KRISHNA YADAV

ROLL NO: S124

SUBJECT: DATA STRUCTURES



Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir

M.V. College

Of Science & Commerce

try:

```
size = int(size_entry.get())
data = list(map(int, data_entry.get().split()))
except ValueError:
    messagebox.showerror("Error", "Enter valid numbers.")
return
```

```
table = [None] * size
```

for value in data:

```
idx = original = value % size
while table[idx] is not None:
    idx = (idx + 1) % size
    if idx == original:
        idx = None
        break
if idx is not None:
    table[idx] = value
```

Show result

```
result.delete(0, tk.END)
for i, v in enumerate(table):
    result.insert(tk.END, f"Index {i}: {v}")
```

```
root = tk.Tk()
```

```
root.title("Linear Probing Hash Table")
```

```
tk.Label(root, text="Table size:").grid(row=0, column=0, padx=5, pady=5)
```

```
size_entry = tk.Entry(root, width=10)
```

```
size_entry.grid(row=0, column=1, padx=5, pady=5)
```

```
tk.Label(root, text="Data (space-separated):").grid(row=1, column=0, padx=5, pady=5)
```

```
data_entry = tk.Entry(root, width=30)
```

```
data_entry.grid(row=1, column=1, padx=5, pady=5)
```

```
tk.Button(root, text="Insert", command=insert_all).grid(row=2, column=0, columnspan=2,
pady=10)
```

```
result = tk.Listbox(root, width=40, height=15)
```

```
result.grid(row=3, column=0, columnspan=2, padx=5, pady=5)
```

NAME: KRISHNA YADAV

ROLL NO: S124

SUBJECT: DATA STRUCTURES



Laxmi Charitable Trust's Sheth L.U.J College of Arts & Sir M.V. College Of Science & Commerce

root.mainloop()

OUTPUT:

```
*IDLE Shell 3.14.2*
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/KRISHNA/Desktop/SYCS 27-28/MJ7 Data Structures/Practical 9 linear probing (hash) , Breadth First Tree and Depth First Tree using Graph/9A.py
Enter size of hash table: 12
Enter data (space-separated numbers): 12

Final Hash Table:
Index 0: 12
Index 1: None
Index 2: None
Index 3: None
Index 4: None
Index 5: None
Index 6: None
Index 7: None
Index 8: None
Index 9: None
Index 10: None
Index 11: None
>>>
= RESTART: C:/Users/KRISHNA/Desktop/SYCS 27-28/MJ7 Data Structures/Practical 9 linear probing (hash) , Breadth First Tree and Depth First Tree using Graph/9A GUI.py

Linear Probing Hash Table
Table size: 10
Data (space-separated): 4
Insert

Index 0: None
Index 1: None
Index 2: None
Index 3: None
Index 4: None
Index 5: None
Index 6: None
Index 7: None
Index 8: None
Index 9: 4
```

9. B. Write a Python program to implement Breadth First Tree using a graph.

CODE:

```
from collections import deque, defaultdict
```

```
class Graph:
    def __init__(self):
        self.graph = defaultdict(list)
```

```
    def add_vertex(self, vertex):
        if vertex not in self.graph:
```

NAME: KRISHNA YADAV

ROLL NO: S124

SUBJECT: DATA STRUCTURES



Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir

M.V. College

Of Science & Commerce

```
self.graph[vertex] = []
```

```
def add_edge(self, vertex1, vertex2):
    if vertex1 in self.graph and vertex2 in self.graph:
        self.graph[vertex1].append(vertex2)
        self.graph[vertex2].append(vertex1)

def bfs_tree(self, start):
    """Perform BFS and return the Breadth-First Tree as a dictionary."""
    visited = set()
    bfs_tree = defaultdict(list)
    queue = deque([start])
    visited.add(start)
    while queue:
        current = queue.popleft()
        for neighbor in self.graph[current]:
            if neighbor not in visited:
                visited.add(neighbor)
                bfs_tree[current].append(neighbor)
                queue.append(neighbor)
    return bfs_tree

def display(self):
    for vertex, edges in self.graph.items():
        print(f"{vertex}: {edges}")

def print_bfs_tree(bfs_tree, start):
    """Print the Breadth-First Tree."""
    print(f"Breadth-First Tree starting from {start}:")
    for vertex in bfs_tree:
        print(f"{vertex}: {bfs_tree[vertex]}")
```

Example usage

```
if __name__ == "__main__":
```

```
    g = Graph()
```

```
    # Add vertices
```

```
    for v in ['A', 'B', 'C', 'D', 'E', 'F']:
```

```
        g.add_vertex(v)
```

NAME: KRISHNA YADAV

ROLL NO: S124

SUBJECT: DATA STRUCTURES



Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir

M.V. College

Of Science & Commerce

```
# Add edges
```

```
g.add_edge('A', 'B')
```

```
g.add_edge('A', 'C')
```

```
g.add_edge('B', 'D')
```

```
g.add_edge('B', 'E')
```

```
g.add_edge('C', 'F')
```

```
print("Graph:")
```

```
g.display()
```

```
# Perform BFS and get the Breadth-First Tree
```

```
start_vertex = 'A'
```

```
bfs_tree = g.bfs_tree(start_vertex)
```

```
print_bfs_tree(bfs_tree, start_vertex)
```

GUI CODE:

```
import tkinter as tk
```

```
from tkinter import messagebox
```

```
from collections import deque, defaultdict
```

```
def run_bfs():
```

```
    try:
```

```
        vertices = [v.strip() for v in vertex_entry.get().split(",") if v.strip()]
```

```
        edge_pairs = [e.strip() for e in edge_entry.get().split(",") if e.strip()]
```

```
        start = start_entry.get().strip()
```

```
        graph = defaultdict(list)
```

```
        for v in vertices:
```

```
            graph[v] = []
```

```
        for pair in edge_pairs:
```

```
            a, b = pair.split("-")
```

```
            a, b = a.strip(), b.strip()
```

```
            graph[a].append(b)
```

```
            graph[b].append(a)
```

```
        if start not in graph:
```

```
            raise ValueError("Start vertex not in graph.")
```

NAME: KRISHNA YADAV

ROLL NO: S124

SUBJECT: DATA STRUCTURES



Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir

M.V. College

Of Science & Commerce

```
visited = {start}
tree = defaultdict(list)
queue = deque([start])
while queue:
    current = queue.popleft()
    for neighbor in graph[current]:
        if neighbor not in visited:
            visited.add(neighbor)
            tree[current].append(neighbor)
            queue.append(neighbor)

result.delete(0, tk.END)
result.insert(tk.END, f"BFS Tree from {start}:")
for v in tree:
    result.insert(tk.END, f"{v}: {tree[v]}")

except Exception as e:
    messagebox.showerror("Error", f"Invalid input.\n{e}")
```

```
root = tk.Tk()
root.title("BFS Tree")
```

```
tk.Label(root, text="Vertices (A,B,C):").grid(row=0, column=0, padx=5, pady=5)
vertex_entry = tk.Entry(root, width=30)
vertex_entry.insert(0, "A,B,C,D,E,F")
vertex_entry.grid(row=0, column=1, padx=5, pady=5)
```

```
tk.Label(root, text="Edges (A-B,A-C):").grid(row=1, column=0, padx=5, pady=5)
edge_entry = tk.Entry(root, width=30)
edge_entry.insert(0, "A-B,A-C,B-D,B-E,C-F")
edge_entry.grid(row=1, column=1, padx=5, pady=5)
```

```
tk.Label(root, text="Start vertex:").grid(row=2, column=0, padx=5, pady=5)
start_entry = tk.Entry(root, width=10)
start_entry.insert(0, "A")
start_entry.grid(row=2, column=1, sticky="w", padx=5, pady=5)
```

```
tk.Button(root, text="Run BFS", command=run_bfs).grid(row=3, column=0, columnspan=2,
pady=10)
```

NAME: KRISHNA YADAV
ROLL NO: S124
SUBJECT: DATA STRUCTURES



Laxmi Charitable Trust's Sheth L.U.J College of Arts & Sir M.V. College Of Science & Commerce

```
result = tk.Listbox(root, width=40, height=12)
result.grid(row=4, column=0, columnspan=2, padx=5, pady=5)

root.mainloop()
```

OUTPUT:

```
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: C:/Users/KRISHNA/Desktop/SYCS 27-28/MJ7 Data Structures/Practical 9 linear probing (hash) , Breadth First Tree and Depth First Tree using Graph/9B.py
Graph:
A: ['B', 'C']
B: ['A', 'D', 'E']
C: ['A', 'F']
D: ['B']
E: ['B']
F: ['C']
Breadth-First Tree starting from A:
A: ['B', 'C']
B: ['D', 'E']
C: ['F']

>>> = RESTART: C:/Users/KRISHNA/Desktop/SYCS 27-28/MJ7 Data Structures/Practical 9 linear probing (hash) , Breadth First Tree and Depth First Tree using Graph/9B GUI.py

BFS Tree
Vertices (A,B,C): A,B,C,D,E,F
Edges (A-B,A-C): A-B,A-C,B-D,B-E,C-F
Start vertex: A
Run BFS

BFS Tree from A:
A: ['B', 'C']
B: ['D', 'E']
C: ['F']
```

9. C. Write a Python program to implement Depth First Tree using Graph

CODE:

```
import matplotlib.pyplot as plt
import networkx as nx
from collections import defaultdict
```

NAME: KRISHNA YADAV

ROLL NO: S124

SUBJECT: DATA STRUCTURES



Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir

M.V. College

Of Science & Commerce

```
class Graph:
    def __init__(self):
        self.graph = defaultdict(list)

    def add_vertex(self, vertex):
        if vertex not in self.graph:
            self.graph[vertex] = []

    def add_edge(self, vertex1, vertex2):
        if vertex1 in self.graph and vertex2 in self.graph:
            self.graph[vertex1].append(vertex2)
            self.graph[vertex2].append(vertex1)

    def dfs_tree(self, start):
        """Perform DFS and return the Depth-First Tree as a dictionary."""
        visited = set()
        dfs_tree = defaultdict(list)

        def dfs(v):
            visited.add(v)
            for neighbor in self.graph[v]:
                if neighbor not in visited:
                    dfs_tree[v].append(neighbor)
                    dfs(neighbor)

        dfs(start)
        return dfs_tree

    def display(self):
        for vertex, edges in self.graph.items():
            print(f"{vertex}: {edges}")

    def visualize(self, dfs_tree=None):
        """Visualize the graph and optionally the DFS tree."""
        G = nx.Graph(self.graph)
        pos = nx.spring_layout(G)

        plt.figure(figsize=(12, 6))
```

NAME: KRISHNA YADAV
ROLL NO: S124
SUBJECT: DATA STRUCTURES



Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir

M.V. College

Of Science & Commerce

```
# Draw the original graph
plt.subplot(121)
nx.draw(G, pos, with_labels=True, node_color='lightblue', edge_color='gray',
        node_size=1000, font_size=15, font_weight='bold')
plt.title("Graph")

# Draw the DFS Tree
if dfs_tree:
    T = nx.DiGraph(dfs_tree)
    pos_tree = nx.spring_layout(T)
    plt.subplot(122)
    nx.draw(T, pos_tree, with_labels=True, node_color='lightgreen', edge_color='blue',
            node_size=1000, font_size=15, font_weight='bold', arrows=True)
    plt.title("DFS Tree")

plt.show()

# Example usage
if __name__ == "__main__":
    g = Graph()

    # Add vertices
    for v in ['A', 'B', 'C', 'D', 'E', 'F']:
        g.add_vertex(v)

    # Add edges
    g.add_edge('A', 'B')
    g.add_edge('A', 'C')
    g.add_edge('B', 'D')
    g.add_edge('B', 'E')
    g.add_edge('C', 'F')

    print("Graph:")
    g.display()

    # Perform DFS and get the Depth-First Tree
    start_vertex = 'A'
    dfs_tree = g.dfs_tree(start_vertex)

    # Visualize the graph and the DFS tree
```

NAME: KRISHNA YADAV
ROLL NO: S124
SUBJECT: DATA STRUCTURES



Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir

M.V. College

Of Science & Commerce

g.visualize(DFS_tree)

GUI CODE:

```
import tkinter as tk
from tkinter import messagebox
from collections import defaultdict
import matplotlib.pyplot as plt
import networkx as nx
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
```

```
def runDFS():
    try:
        vertices = [v.strip() for v in vertex_entry.get().split(",") if v.strip()]
        edge_pairs = [e.strip() for e in edge_entry.get().split(",") if e.strip()]
        start = start_entry.get().strip()

        graph = defaultdict(list)
        for v in vertices:
            graph[v] = []

        for pair in edge_pairs:
            a, b = pair.split("-")
            a, b = a.strip(), b.strip()
            graph[a].append(b)
            graph[b].append(a)

        if start not in graph:
            raise ValueError("Start vertex not in graph.")

        visited = set()
        tree = defaultdict(list)

        def dfs(v):
            visited.add(v)
            for neighbor in graph[v]:
                if neighbor not in visited:
                    tree[v].append(neighbor)
                    dfs(neighbor)

        dfs(start)
```

NAME: KRISHNA YADAV

ROLL NO: S124

SUBJECT: DATA STRUCTURES



Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir

M.V. College

Of Science & Commerce

```
# Text result
result.delete(0, tk.END)
result.insert(tk.END, f"DFS Tree from {start}:")
for v in tree:
    result.insert(tk.END, f"{v}: {tree[v]}")

# Clear old plot
for widget in plot_frame.winfo_children():
    widget.destroy()

# Draw graph + DFS tree
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(8, 4))

G = nx.Graph(graph)
pos = nx.spring_layout(G)
nx.draw(G, pos, with_labels=True, node_color='lightblue',
        edge_color='gray', node_size=800, font_size=12,
        font_weight='bold', ax=ax1)
ax1.set_title("Graph")

T = nx.DiGraph(tree)
pos_tree = nx.spring_layout(T)
nx.draw(T, pos_tree, with_labels=True, node_color='lightgreen',
        edge_color='blue', node_size=800, font_size=12,
        font_weight='bold', arrows=True, ax=ax2)
ax2.set_title("DFS Tree")

canvas = FigureCanvasTkAgg(fig, master=plot_frame)
canvas.draw()
canvas.get_tk_widget().pack(fill="both", expand=True)

except Exception as e:
    messagebox.showerror("Error", f"Invalid input.\n{e}")

root = tk.Tk()
root.title("DFS Tree")

tk.Label(root, text="Vertices (A,B,C):").grid(row=0, column=0, padx=5, pady=5)
vertex_entry = tk.Entry(root, width=30)

NAME: KRISHNA YADAV
ROLL NO: S124
SUBJECT: DATA STRUCTURES
```



Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir

M.V. College

Of Science & Commerce

```
vertex_entry.insert(0, "A,B,C,D,E,F")
vertex_entry.grid(row=0, column=1, padx=5, pady=5)

tk.Label(root, text="Edges (A-B,A-C):").grid(row=1, column=0, padx=5, pady=5)
edge_entry = tk.Entry(root, width=30)
edge_entry.insert(0, "A-B,A-C,B-D,B-E,C-F")
edge_entry.grid(row=1, column=1, padx=5, pady=5)

tk.Label(root, text="Start vertex:").grid(row=2, column=0, padx=5, pady=5)
start_entry = tk.Entry(root, width=10)
start_entry.insert(0, "A")
start_entry.grid(row=2, column=1, sticky="w", padx=5, pady=5)

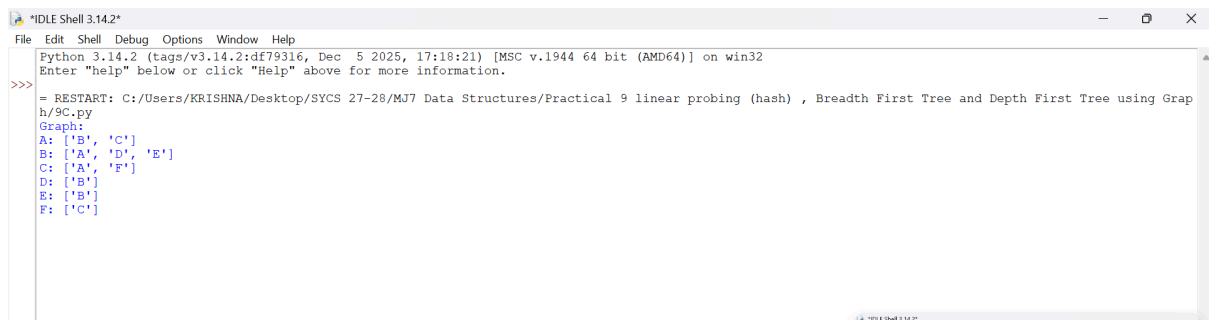
tk.Button(root, text="Run DFS", command=run_dfs).grid(row=3, column=0, columnspan=2,
pady=10)

result = tk.Listbox(root, width=40, height=8)
result.grid(row=4, column=0, columnspan=2, padx=5, pady=5)

plot_frame = tk.Frame(root)
plot_frame.grid(row=5, column=0, columnspan=2, padx=5, pady=5)

root.mainloop()
```

OUTPUT:



```
*IDLE Shell 3.14.2*
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

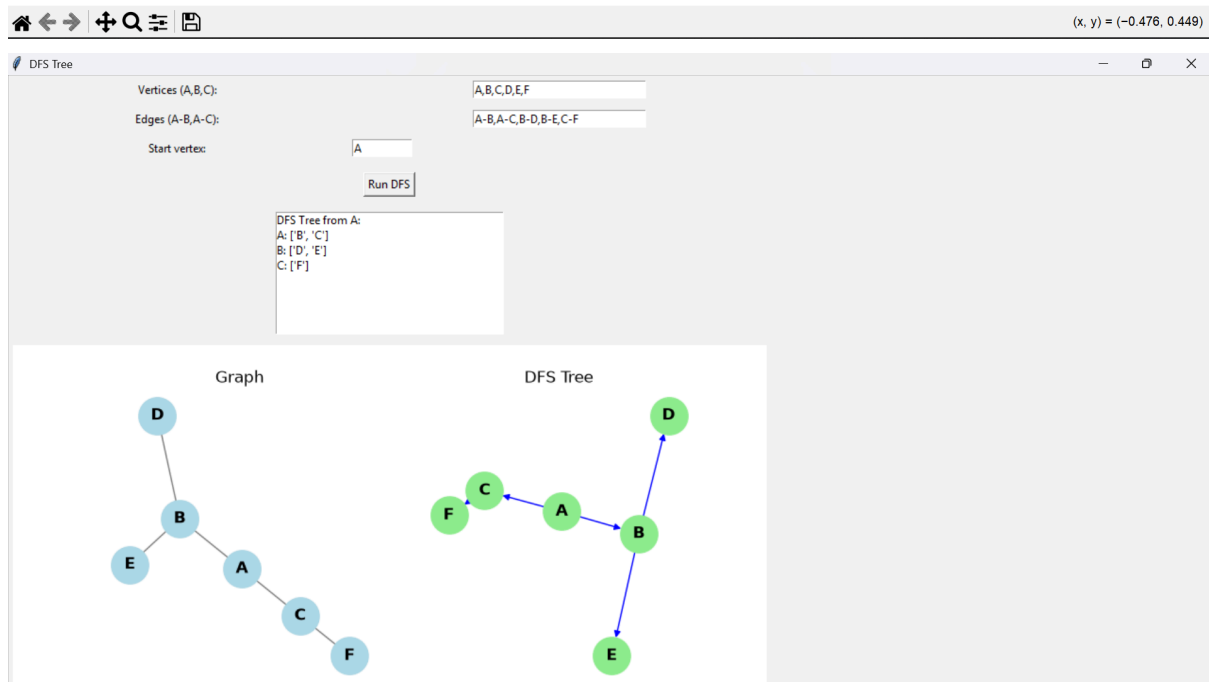
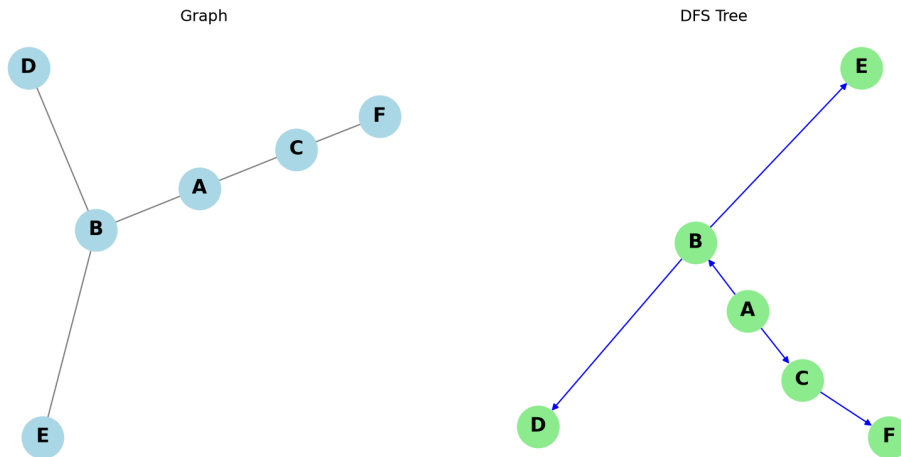
>>>
= RESTART: C:/Users/KRISHNA/Desktop/SYCS 27-28/MJ7 Data Structures/Practical 9 linear probing (hash) , Breadth First Tree and Depth First Tree using Grap
h/9C.py
Graph:
A: ['B', 'C']
B: ['A', 'D', 'E']
C: ['A', 'F']
D: ['B']
E: ['B']
F: ['C']
```

NAME: KRISHNA YADAV
ROLL NO: S124
SUBJECT: DATA STRUCTURES



Laxmi Charitable Trust's Sheth L.U.J College of Arts & Sir M.V. College Of Science & Commerce

Figure 1



NAME: KRISHNA YADAV
ROLL NO: S124
SUBJECT: DATA STRUCTURES